Session 2.2 — Loops: Making Python Repeat Things 🔁

Session Overview

Detail	Info
Session	2.2
Topic	For Loops, While Loops, Iterating Over Lists, Loop Control (Break, Continue)
Duration	2 hours (teaching) + 10 min break + 20 min Q&A
Environment	Google Colab
Audience	Complete beginners (college students)
Previous Session	2.1 — Variables, Data Types, Math, Print/Input, Conditionals, Boolean Logic

Learning Objectives

By the end of this session, you will be able to:

- ✅ Construct `for` loops to iterate over sequences
- ✅ Write `while` loops with proper termination conditions
- ✅ Iterate over simple lists using loops
- ✅ Use `break` to exit a loop early
- ✅ Use `continue` to skip iterations

What We've Covered So Far

Session	Topics
2.1	Variables, Data Types (int, float, str, bool), Math Operators, Print & Input, Conditionals (if/elif/else), Boolean Logic (and, or, not), Indentation

Session Timeline

Time	Topic	Duration
0:00 - 0:05	Recap of Session 2.1 & Why We Need Loops	5 min
0:05 - 0:35	For Loops & range()	30 min
0:35 - 0:55	Iterating Over Lists	20 min
0:55 - 1:05	☕ Break	10 min
1:05 - 1:35	While Loops	30 min
1:35 - 1:55	Loop Control — Break & Continue	20 min
1:55 - 2:00	Recap & What's Next	5 min
2:00 - 2:20	Q&A / Doubt Clearing	20 min

Part 1: Why Do We Need Loops? (5 min)

🔗 Connecting to Session 2.1

Last session, we learned how to make Python *make decisions* using `if-elif-else`. Our programs could choose different paths based on conditions. That's powerful.

But here's a problem. Let's say you want to print "Hello" 5 times:

```
print("Hello")
print("Hello")
print("Hello")
print("Hello")
print("Hello")
```

That works. But what if you want to print it 100 times? Or 10,000 times? You're not going to write 10,000 lines of code, right?

Or what if you have a list of 50 student names and you want to print each one? Writing 50 separate `print()` statements is madness.

> 🔗 ***The shortcoming of what we know so far**: We can make decisions, but we can't make Python repeat things. Every line runs once and that's it. We need a way to say: "Do this thing… again and again… until I tell you to stop."*

That's exactly what **loops** do.

🤔 What is a Loop?

A loop is an instruction that tells Python: **“Repeat this block of code multiple times.”**

Think of it like this:

```
Without loops:                    With loops:
──────────────                    ───────────
print("Hello")                    Repeat 5 times:
print("Hello")                        print("Hello")
print("Hello")        →
print("Hello")                    (That's it. 2 lines instead of 5.)
print("Hello")
```

Python gives us two types of loops:

Loop Type	When to use it
`for` loop	When you know HOW MANY times to repeat (or you have a collection to go through)
`while` loop	When you DON’T know how many times — you repeat UNTIL a condition becomes False

Let’s start with the `for` loop.

Part 2: The `for` Loop — “Do This for Each Item” (30 min)

💡 What is a `for` loop?

A `for` loop repeats a block of code **for each item** in a sequence (like a list of numbers, a range, or a string).

🤔 When do we use it?

When you know in advance how many times you want to repeat, or when you have a collection of items to go through one by one.

🏏 Real-Life Analogy

Imagine a cricket coach calling out each player’s name during roll call:

```
Coach has a list: [Rohit, Virat, Dhoni, Bumrah, Jadeja]

For EACH name in the list:
    Call out the name
```

That’s a `for` loop — go through each item, one at a time, and do something with it.

📝 Syntax

```
for variable in sequence:
    # Code to repeat (indented — just like if-else!)
    # This runs once for each item in the sequence
```

Part	What it means
`for`	Keyword that starts the loop
`variable`	A temporary name that holds the current item (changes each iteration)
`in`	Keyword that means “from”
`sequence`	The collection of items to loop through
`:`	Don’t forget the colon!
Indented block	The code that repeats

🔢 `range()` — Generating Numbers to Loop Over

The most common way to use a `for` loop is with `range()`. It generates a sequence of numbers.

`range(n)` — generates numbers from 0 to n-1:

```
for i in range(5):
    print(i)
```

Output:

```
0
1
2
3
4
```

Wait — why does it go from 0 to 4, not 1 to 5? Because Python starts counting from 0. And `range(5)` means “give me 5 numbers, starting from 0.”

```
range(5) generates: 0, 1, 2, 3, 4
                    ↑              ↑
                starts at 0    stops BEFORE 5
```

📋 Three Ways to Use `range()`

Syntax	What it generates	Example
`range(stop)`	0 to stop-1	`range(5)` → 0, 1, 2, 3, 4
`range(start, stop)`	start to stop-1	`range(2, 7)` → 2, 3, 4, 5, 6
`range(start, stop, step)`	start to stop-1, jumping by step	`range(0, 10, 2)` → 0, 2, 4, 6, 8

Examples:

```python
# range(stop) — start from 0
for i in range(4):
    print(i)
# Output: 0, 1, 2, 3

# range(start, stop) — custom start
for i in range(1, 6):
    print(i)
# Output: 1, 2, 3, 4, 5

# range(start, stop, step) — skip numbers
for i in range(0, 10, 2):
    print(i)
# Output: 0, 2, 4, 6, 8

# Counting backwards!
for i in range(5, 0, -1):
    print(i)
# Output: 5, 4, 3, 2, 1
```

🔍 Let's Trace Through a Loop Step by Step

```python
for i in range(3):
    print(f"Iteration {i}: Hello!")
```

What Python does internally:

```
Step 1: range(3) generates → [0, 1, 2]

Step 2: i = 0 → print("Iteration 0: Hello!")
Step 3: i = 1 → print("Iteration 1: Hello!")
Step 4: i = 2 → print("Iteration 2: Hello!")

Step 5: No more items → loop ends
```

Output:

```
Iteration 0: Hello!
Iteration 1: Hello!
Iteration 2: Hello!
```

🧮 Real-Life Example: Multiplication Table

```python
number = 7

print(f"--- Multiplication Table of {number} ---")
for i in range(1, 11):
    result = number * i
    print(f"{number} × {i} = {result}")
```

Output:

```
--- Multiplication Table of 7 ---
7 × 1 = 7
7 × 2 = 14
7 × 3 = 21
...
7 × 10 = 70
```

10 lines of output from just 4 lines of code. That's the power of loops.

🧪 Try It — Sum of First N Numbers

```python
n = int(input("Enter a number: "))
total = 0

for i in range(1, n + 1):
    total = total + i

print(f"Sum of first {n} numbers = {total}")
```

Sample run:

```
Enter a number: 5
Sum of first 5 numbers = 15
```

Trace: `total = 0 + 1 + 2 + 3 + 4 + 5 = 15`

🔤 Looping Over Strings

A string is a sequence of characters, so you can loop over it too:

```python
name = "Python"

for letter in name:
    print(letter)
```

Output:

```
P
y
t
h
o
n
```

Each iteration, `letter` holds one character from the string.

> ⚠️ ***Common Mistakes with `for` loops***
>
> *1. Forgetting the colon: `for i in range(5)` ❌ → `for i in range(5):` ✅*
>
> *2. Wrong indentation inside the loop — same rules as `if-else`*
>
> *3. Expecting `range(5)` to include 5: it goes 0 to 4, NOT 1 to 5*
>
> *4. Forgetting that `range(1, 5)` stops at 4, not 5 — the stop value is always EXCLUDED*
>
> *5. Modifying the loop variable inside the loop: `for i in range(5): i = 10` — this doesn't break the loop, `i` just gets overwritten next iteration*
>
> *6. Using `range(5, 1)` expecting 5,4,3,2 — you need a negative step: `range(5, 1, -1)`*
>
> 🔗 ***Connection**: `for` loops are great when you have a sequence to go through — numbers, strings, lists. But what if you don't have a fixed sequence? What if you want to keep asking the user for input until they type "quit"? You don't know in advance how many times that'll be. That's where **lists** and **while loops** come in. But first, let's learn how to loop over lists.*

Part 3: Iterating Over Lists (20 min)

💡 What is a List?

A list is a collection of items stored in a single variable. Think of it as a **shopping list** — multiple items, in order, all in one place.

```python
# A list of fruits
fruits = ["Apple", "Banana", "Mango", "Orange"]

# A list of marks
marks = [85, 92, 78, 95, 88]

# A list can have mixed types (but usually you keep them the same)
mixed = ["Rahul", 20, True, 5.8]
```

> 📝 ***Note**: We'll cover lists in much more detail in a future session. For now, just know that a list is a collection inside square brackets `[]`, and you can loop over it.*

🔁 Looping Over a List with `for`

This is where `for` loops really shine:

```
fruits = ["Apple", "Banana", "Mango", "Orange"]

for fruit in fruits:
    print(fruit)
```

Output:

```
Apple
Banana
Mango
Orange
```

What Python does:

```
Iteration 1: fruit = "Apple"  → print("Apple")
Iteration 2: fruit = "Banana" → print("Banana")
Iteration 3: fruit = "Mango"  → print("Mango")
Iteration 4: fruit = "Orange" → print("Orange")
Done — no more items.
```

🏏 Real-Life Example: Cricket Team Roll Call

```
players = ["Rohit", "Virat", "Dhoni", "Bumrah", "Jadeja"]

print("🏏 Team Roll Call:")
for player in players:
    print(f"  ✅ {player} is present")

print(f"\nTotal players: {len(players)}")
```

Output:

```
🏏 Team Roll Call:
  ✅ Rohit is present
  ✅ Virat is present
  ✅ Dhoni is present
  ✅ Bumrah is present
  ✅ Jadeja is present

Total players: 5
```

📊 Real-Life Example: Calculating Average Marks

```
marks = [85, 92, 78, 95, 88]
total = 0

for mark in marks:
    total = total + mark

average = total / len(marks)
print(f"Total: {total}")
print(f"Average: {average}")
```

Output:

```
Total: 438
Average: 87.6
```

Trace:

```
mark = 85 → total = 0 + 85 = 85
mark = 92 → total = 85 + 92 = 177
mark = 78 → total = 177 + 78 = 255
mark = 95 → total = 255 + 95 = 350
mark = 88 → total = 350 + 88 = 438
```

🔢 Using `enumerate()` — When You Need the Index Too

Sometimes you want to know WHICH item you're on (the position/index):

```
fruits = ["Apple", "Banana", "Mango", "Orange"]

for index, fruit in enumerate(fruits):
    print(f"{index + 1}. {fruit}")
```

Output:

```
1. Apple
2. Banana
3. Mango
4. Orange
```

📝 ***Note***: *`enumerate()` gives you both the index (starting from 0) and the item. We add 1 to make it human-friendly (1, 2, 3… instead of 0, 1, 2…).*

🧪 Try It — Find the Highest Mark

```
marks = [85, 92, 78, 95, 88]
highest = marks[0]    # Start by assuming the first mark is the highest

for mark in marks:
    if mark > highest:
        highest = mark

print(f"Highest mark: {highest}")
# Output: Highest mark: 95
```

> ⚠️ ***Common Mistakes with List Iteration***
>
> *1. Modifying a list while looping over it — can cause unexpected behavior*
>
> *2. Using the wrong variable name: `for fruit in marks:` — it works, but it's confusing. Name your loop variable to match the list content.*
>
> *3. Forgetting that list indices start at 0: `fruits[1]` is "Banana", not "Apple"*
>
> 🔗 ***Connection**: `for` loops are perfect when you have a collection (list, range, string) to go through. But what if you DON'T have a collection? What if you want to keep doing something until a condition changes? Like: "Keep asking the user for a password until they get it right." You don't know how many attempts they'll need — could be 1, could be 100. `for` loops can't handle this because you'd need to know the count upfront. That's where `while` **loops** come in.*

☕ Break Time! (10 min)

Take a breather. Stretch. Grab some chai. ☕

When we come back, we'll learn the second type of loop — `while` — which keeps going until you tell it to stop.

Part 4: The `while` Loop — "Keep Going Until…" (30 min)

💡 What is a `while` loop?

A `while` loop repeats a block of code **as long as a condition is True**. The moment the condition becomes False, the loop stops.

🤔 When do we use it?

When you DON'T know in advance how many times you need to repeat. You just know the CONDITION for when to stop.

🤔 Why do we need it?

Because `for` loops need a fixed sequence — you must know the count or have a collection. But many real-world situations don't have a fixed count:
- "Keep asking for a password until the user gets it right" — how many attempts? Unknown.
- "Keep withdrawing money until the balance is zero" — how many withdrawals? Depends on amounts.
- "Keep playing the game until the player says quit" — how many rounds? Unknown.

🎮 Real-Life Analogy

Think of a video game:

```
WHILE the player has lives remaining:
    Play the next round
    If player dies, lose a life

Game Over (lives = 0, condition is False, loop stops)
```

You don't know how many rounds the player will survive. The loop just keeps going until the condition fails.

📝 Syntax

```
while condition:
    # Code to repeat (indented)
    # Something inside MUST eventually make the condition False
    # Otherwise the loop runs FOREVER (infinite loop!)
```

🔍 How It Works — Step by Step

```
count = 1

while count <= 5:
    print(f"Count: {count}")
    count = count + 1

print("Loop finished!")
```

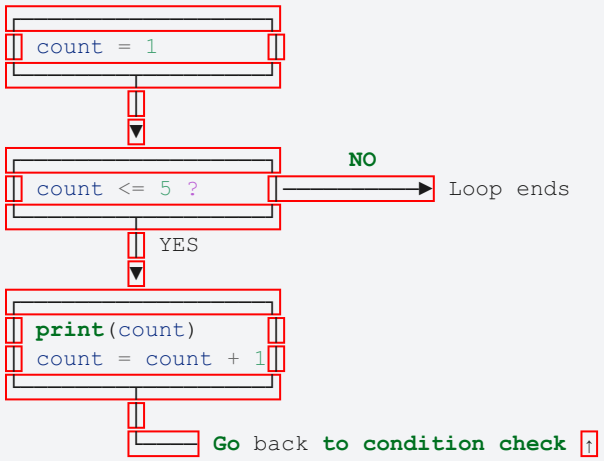
Trace:

```
Step 1: count = 1 → Is 1 <= 5? YES → print "Count: 1" → count = 2
Step 2: count = 2 → Is 2 <= 5? YES → print "Count: 2" → count = 3
Step 3: count = 3 → Is 3 <= 5? YES → print "Count: 3" → count = 4
Step 4: count = 4 → Is 4 <= 5? YES → print "Count: 4" → count = 5
Step 5: count = 5 → Is 5 <= 5? YES → print "Count: 5" → count = 6
Step 6: count = 6 → Is 6 <= 5? NO  → EXIT loop
→ print "Loop finished!"
```

Output:

```
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Loop finished!
```

The flow:



🔐 Real-Life Example: Password Checker

```python
correct_password = "python123"

password = input("Enter password: ")

while password != correct_password:
    print("❌ Wrong password! Try again.")
    password = input("Enter password: ")

print("✅ Login successful! Welcome!")
```

Sample run:

```
Enter password: hello
❌ Wrong password! Try again.
Enter password: 12345
❌ Wrong password! Try again.
Enter password: python123
✅ Login successful! Welcome!
```

The loop kept running because the condition `password != correct_password` was True. The moment the user typed the right password, the condition became False, and the loop stopped.

🏧 Real-Life Example: ATM Withdrawal

```
balance = 1000

while balance > 0:
    print(f"\n💰 Current balance: ₹{balance}")
    amount = int(input("Enter withdrawal amount (0 to quit): "))

    if amount == 0:
        print("Thank you! Goodbye.")
        break    # We'll learn about break soon!
    elif amount > balance:
        print("❌ Insufficient balance!")
    else:
        balance = balance - amount
        print(f"✅ ₹{amount} withdrawn successfully.")

if balance == 0:
    print("⚠️ Your balance is now ₹0.")
```

🚨 The Infinite Loop — The Biggest Danger

What happens if the condition NEVER becomes False?

```
# ⚠️ DANGER — DO NOT RUN THIS (or be ready to stop it!)
count = 1

while count <= 5:
    print(f"Count: {count}")
    # Oops! We forgot to update count!
    # count stays 1 forever → 1 <= 5 is ALWAYS True
    # This loop NEVER stops!
```

This is called an **infinite loop** — it runs forever until you manually stop it (Ctrl+C in terminal, or stop button in Colab).

How to avoid infinite loops:

```
✅ Rule: Something inside the while loop MUST change the condition
         so that it eventually becomes False.
```

```
# ✅ SAFE — count increases each time, eventually count > 5
count = 1
while count <= 5:
    print(count)
    count = count + 1     # ← This is what makes the loop stop!

# ❌ INFINITE — nothing changes count
count = 1
while count <= 5:
    print(count)
    # count never changes → loop never ends
```

📋 `for` vs `while` — When to Use Which?

Situation	Use	Why
Loop a fixed number of times	`for`	You know the count: `for i in range(10)`
Go through a list/string	`for`	You have a collection: `for item in list`
Repeat until a condition changes	`while`	You don't know the count: `while password != correct`
Keep asking for user input	`while`	Unknown number of attempts
Countdown or count up by a pattern	`for` with `range()`	`range(10, 0, -1)` for countdown

> 💡 ***Key Idea***: *If you can use a `for` loop, prefer it — it's safer (no risk of infinite loops) and cleaner. Use `while` only when you genuinely don't know how many iterations you need.*

🧪 Try It — Guessing Game

```python
import random

secret = random.randint(1, 10)
guess = 0
attempts = 0

print("🎯 I'm thinking of a number between 1 and 10...")

while guess != secret:
    guess = int(input("Your guess: "))
    attempts = attempts + 1

    if guess < secret:
        print("📈 Too low! Try higher.")
    elif guess > secret:
        print("📉 Too high! Try lower.")

print(f"🎉 Correct! You got it in {attempts} attempts!")
```

> ⚠️ ***Common Mistakes with `while` loops***
>
> *1. Forgetting to update the condition variable → infinite loop*
>
> *2. Off-by-one errors: `while count < 5` runs 4 times (0,1,2,3), `while count <= 5` runs 5 times (0,1,2,3,4,5 if starting from 0)*
>
> *3. Using `=` instead of `==` in the condition: `while count = 5` ❌ → `while count == 5` ✅*
>
> *4. Putting the update BEFORE the print — changes the order of output*
>
> *5. Not having an exit condition for user-input loops — always give the user a way out (like "type 0 to quit")*

> 🔗 ***Connection**: Both `for` and `while` loops run from start to finish — every iteration, every item. But what if you want to STOP early? Like searching for a name in a list — once you find it, why keep looking? Or what if you want to SKIP certain items? Like printing only even numbers and skipping odd ones. That's where `break` and `continue` come in.*

Part 5: Loop Control — `break` and `continue` (20 min)

🤔 The Problem

Right now, our loops are "all or nothing" — they run every single iteration from start to finish. But sometimes we need more control:

- **Stop early**: "I found what I was looking for, no need to keep looping"
- **Skip one iteration**: "This item doesn't meet my criteria, skip it and move to the next"

Python gives us two keywords for this: `break` and `continue`.

💡 `break` — "Stop the Loop Right Now!"

What is it? `break` immediately exits the loop, no matter how many iterations are left.

When do we use it? When you've found what you need and there's no point continuing.

Why do we need it? Without `break`, the loop would keep running even after you've achieved your goal — wasting time and effort.

Real-life analogy:

> *You're searching for your keys in 5 drawers. You find them in drawer 3. Do you keep opening drawers 4 and 5? No! You **break** out of the search.*

Example — Finding a Name:

```python
students = ["Amit", "Priya", "Rahul", "Sneha", "Vikram"]
search = "Rahul"

for student in students:
    print(f"Checking: {student}")
    if student == search:
        print(f"✅ Found {search}!")
        break     # Stop searching — we found them!

# Output:
# Checking: Amit
# Checking: Priya
# Checking: Rahul
# ✅ Found Rahul!
# (Sneha and Vikram are never checked — loop stopped early)
```

Trace:

```
Iteration 1: student = "Amit"  → "Amit" == "Rahul"? NO  → continue
Iteration 2: student = "Priya" → "Priya" == "Rahul"? NO → continue
Iteration 3: student = "Rahul" → "Rahul" == "Rahul"? YES → print "Found!" → BREAK!
(Loop ends. "Sneha" and "Vikram" are never reached.)
```

`break` **with** `while` **loop:**

```python
# Keep asking until user types "quit"
while True:     # This looks like an infinite loop...
    command = input("Enter command (type 'quit' to exit): ")

    if command == "quit":
        print("Goodbye! 👋")
        break     # ...but break saves us!

    print(f"You entered: {command}")
```

Sample run:

```
Enter command (type 'quit' to exit): hello
You entered: hello
Enter command (type 'quit' to exit): test
You entered: test
Enter command (type 'quit' to exit): quit
Goodbye! 👋
```

> 💡 ***Key Idea***: `while True` + `break` *is a common pattern. The* `while True` *creates a loop that runs forever, and* `break` *is the only way out. This is useful when the exit condition is checked INSIDE the loop, not at the top.*

💡 `continue` — "Skip This One, Move to the Next"

What is it? `continue` skips the REST of the current iteration and jumps to the NEXT one.

When do we use it? When certain items don't meet your criteria and you want to skip them without stopping the entire loop.

Why do we need it? Without `continue`, you'd have to wrap everything in an `if-else`, which gets messy. `continue` keeps the code clean.

Real-life analogy:

> *You're grading papers. You come across a blank paper (student was absent). You don't grade it — you just* ***skip*** *it and move to the next paper. You don't stop grading entirely.*

Example — Print Only Even Numbers:

```
for i in range(1, 11):
    if i % 2 != 0:     # If the number is odd...
        continue        # ...skip it!
    print(i)
```

Output:

```
2
4
6
8
10
```

Trace:

```
i = 1  → 1 % 2 != 0? YES (odd) → SKIP (continue)
i = 2  → 2 % 2 != 0? NO (even) → print 2
i = 3  → 3 % 2 != 0? YES (odd) → SKIP (continue)
i = 4  → 4 % 2 != 0? NO (even) → print 4
i = 5  → 5 % 2 != 0? YES (odd) → SKIP (continue)
...and so on
```

🤔 `continue` vs Just Using `if`

You might think: “Can’t I just use `if` instead of `continue`?” Yes, but compare:

```
# Using continue (cleaner when there's lots of code after the check)
for student in students:
    if student.absent:
        continue
    # 10 lines of grading logic here...
    # All of this is skipped for absent students

# Using if-else (works, but everything is nested)
for student in students:
    if not student.absent:
        # 10 lines of grading logic here...
        # All indented one extra level
```

With `continue`, you handle the “skip” case early and keep the main logic at the normal indentation level. It’s cleaner when the main logic is long.

📋 `break` vs `continue` — Side by Side

```
break:                                  continue:
━━━━━━━━━━━━                            ━━━━━━━━━━━━━
Loop iteration 1 ✅                      Loop iteration 1 ✅
Loop iteration 2 ✅                      Loop iteration 2 ✅
Loop iteration 3 → BREAK! 🛑            Loop iteration 3 → SKIP! ⏭️
(Loop ends completely)                   Loop iteration 4 ✅
                                         Loop iteration 5 ✅
                                         (Loop continues normally)
```

Keyword	What it does	Analogy
`break`	Exits the ENTIRE loop immediately	Walking out of a movie theater
`continue`	Skips the CURRENT iteration, moves to next	Skipping a song on a playlist

🛒 Real-Life Example: Processing Orders

```python
orders = [150, 0, 300, -50, 200, 175]

print("Processing orders:")
for order in orders:
    if order <= 0:
        print(f"  ⏭️ Skipping invalid order: ₹{order}")
        continue    # Skip invalid orders

    print(f"  ✅ Processing order: ₹{order}")

print("All valid orders processed!")
```

Output:

```
Processing orders:
  ✅ Processing order: ₹150
  ⏭️ Skipping invalid order: ₹0
  ✅ Processing order: ₹300
  ⏭️ Skipping invalid order: ₹-50
  ✅ Processing order: ₹200
  ✅ Processing order: ₹175
All valid orders processed!
```

🧪 Try It — First Negative Number Finder

```python
numbers = [10, 25, 8, -3, 15, -7, 20]

for num in numbers:
    if num < 0:
        print(f"First negative number found: {num}")
        break
else:
    print("No negative numbers found!")

# Output: First negative number found: -3
```

📝 ***Note**: That `else` after the `for` loop is a special Python feature — the `else` block runs ONLY if the loop completed without hitting a `break`. If `break` was triggered, the `else` is skipped. This is useful for "search" patterns.*

⚠️ ***Common Mistakes with `break` and `continue`***

1. Using `break` when you meant `continue` (or vice versa) — `break` STOPS the loop, `continue` just SKIPS one iteration

2. Putting code AFTER `break` or `continue` in the same block — that code will never run

3. Using `break` outside a loop — `SyntaxError`! `break` only works inside `for` or `while`

4. Forgetting that `break` only exits the INNERMOST loop (important when we learn nested loops later)

🎯 Session Recap — What We Learned Today

Let's trace the full journey:

```
Last session: We could make decisions (if-else)
    │   └── But we couldn't REPEAT things
    ▼
for loop → Repeat a fixed number of times / go through a sequence
    │   └── Uses range() for numbers, works on strings and lists
    │   └── But what if we don't know HOW MANY times?
    ▼
Iterating over lists → Go through collections item by item
    │   └── Great for processing data
    │   └── But what about unknown-length repetition?
    ▼
while loop → Keep going UNTIL a condition becomes False
    │   └── Perfect for user input, games, unknown iterations
    │   └── But loops always run start to finish...
    ▼
break → Exit a loop early when you've found what you need
continue → Skip one iteration and move to the next
    │
    ▼
🎉 You can now make Python repeat things intelligently!
```

💡 Key Takeaways

Concept	One-liner
`for` loop	Repeat for each item in a sequence: `for i in range(5):`
`range()`	Generate numbers: `range(stop)`, `range(start, stop)`, `range(start, stop, step)`
List iteration	Loop through items: `for fruit in fruits:`
`while` loop	Repeat while condition is True: `while count < 10:`
Infinite loop	When the condition never becomes False — always update your variable!
`break`	Exit the entire loop immediately
`continue`	Skip current iteration, move to next
`for` vs `while`	Know the count? → `for`. Don't know? → `while`

🔮 What's Coming Next?

We now know how to repeat things. But what if you want to reuse a block of code in multiple places? Like a "calculate average" block that you use for marks, for temperatures, for prices? You'd have to copy-paste the same code everywhere.

Next session, we'll learn **Functions** — how to package code into reusable blocks that you can call by name. Think of them as recipes — write once, use many times. 🚀

Session 2.2 — Loops: Making Python Repeat Things | Created for Batch 2, Module 1